

Exp. No: 04

IMPLEMENTATION OF KNAPSACK USING GREEDY TECHNIQUE.

Date:

17/3/23

AIM:

To write a C++ program to implement knapsack using greedy technique.

ALGORITHM:

Greedy Knapsack (m, n)

// $P[1:n]$ and $w[1:n]$ contains the profits and weights respectively

// of the n objects ordered such that $P[i]/w[i] \geq P[i+1]/w[i+1]$

// m is the knapsack size and $x[1:n]$ is the solution vector.

{ for $i = 1$ to n do $x[i] := 0.0$;

// initialize x

$u := m$;

for $i = 1$ to n do

{ if $(w[i] > u)$ then break;

$x[i] := 1.0$; $u := u - w[i]$;

} if $(i \leq n)$ then $x[i] := u/w[i]$;

}

NAME:KEERTHANAA B; REG.NO:21TD0035; YEAR/SEM/SEC:
Y2/S4/A

KNAP SACK

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
float w[100],v[100],x[100],wt,wgt[100];
class greed
{
private:
int i,j,n;
float r[100],f,total,t,rat,pro,wei,op,opti;
public:
void getdata();
void ratio();
void profit();
void weight();
void optimal();
};
void greedy(int n)
{
int wgt,i;
for(i=0;i<n;i++)
x[i]=0;
wgt=i=0;
while(wgt<wt)
{
if(wgt+w[i]<=wt)
{
x[i]=1;
wgt+=w[i];
}
else

{
x[i]=(wt-wgt)/w[i];
wgt+=(x[i]*w[i]);
}
i++;
}
```



```

}
void greed::getdata()
{

cout<<"\n enter no of object----";
cin>>n;
cout<<"\n enter the capacity of the bag--";
cin>>wt;
for(i=0;i<n;i++)
{
cout<<"weight=";
cin>>w[i];
cout<<"profit=";
cin>>v[i];
r[i]=v[i]/w[i];
}
}

void greed::ratio()
{
cout<<"GREEDY BY RATIO\n";
for(i=0;i<n;i++)
for(j=i+1;j<n;j++)
if(r[i]<r[j])
{
t=w[i];w[i]=w[j];w[j]=t;
t=v[i];v[i]=v[j];v[j]=t;
t=r[i];r[i]=r[j];r[j]=t;
}
greedy(n);
for(i=0;i<n;i++)
{
cout<<"\n weight="<<w[i];
cout<<"\n profit="<<v[i];
cout<<"\n ratio="<<r[i];
cout<<"\n fraction="<<x[i];
total+=(x[i]*v[i]);
}
cout<<"\n feasible profit="<<total;
rat=total;
getch();
total=0;
}

void greed::profit()
{
cout<<"\n GREEDY BY PROFIT";

```

```
for(i=0;i<n;i++)  
for(j=i+1;j<n;j++)  
if(v[i]<v[j])
```

```
{  
t=w[i];w[i]=w[j];w[j]=t;  
t=v[i];v[i]=v[j];v[j]=t;  
t=r[i];r[i]=r[j];r[j]=t;  
}  
greedy(n);  
for(i=0;i<n;i++)  
{  
cout<<"\n weight="<<w[i];  
cout<<"\n profit="<<v[i];  
cout<<"\n ratio ="<<r[i];  
cout<<"\n fraction="<<x[i];  
total+=(x[i]*v[i]);  
}  
cout<<"\n feasible profit="<<total;  
pro=total;  
getch();  
total=0;  
}  
void greed::weight()  
{  
cout<<"\n GREEDY BY WEIGHT";  
for(i=0;i<n;i++)  
for(j=j+1;j<n;j++)  
if(w[i]<w[j])  
{  
t=w[i];w[i]=w[j];w[j]=t;  
t=v[i];v[i]=v[j];v[j]=t;  
t=r[i];r[i]=r[j];r[j]=t;  
}  
greedy(n);  
for(i=0;i<n;i++)  
{  
cout<<"\n weight="<<w[i];  
cout<<"\n profit="<<v[i];  
cout<<"\n ratio="<<r[i];  
cout<<"\n fraction="<<x[i];  
total+=(x[i]*v[i]);  
}  
cout<<"\n feasible profit="<<total;  
wei=total;
```



```
getch();  
}  
void greed::optimal()
```

```
{  
if(rat>pro)  
op=rat;  
else  
op=pro;  
if(op>wei)  
opti=op;  
else  
opti=wei;  
cout<<"\n optimal solution is ....."<<opti;  
getch();  
}
```

```
void main()  
{  
clrscr();  
greed s;  
cout<<"\n KNAP SACK PROBLEM..";  
s.getdata();  
s.ratio();  
s.profit();  
s.weight();  
s.optimal();  
getch();  
}
```

OUTPUT:

```
C:\C++\BIN\TC.EXE
KNAP SACK PROBLEM..
enter no of object----2

enter the capacity of the bag--20
weight=40
profit=10
weight=55
profit=15
GREEDY BY RATION
weight=40
profit=15
ratio=0.272727
fraction=0.5
weight=40
profit=10
ratio=0.25
fraction=0
feasible profit=7.5
GREEDY BY PROFIT
weight=40
profit=15
ratio=0.272727
fraction=0.5
weight=40
profit=10
ratio=0.25
fraction=0
feasible profit=7.5
GREEDY BY WEIGHT
weight=40
profit=15
ratio=0.272727
fraction=0.5
weight=40
profit=10
ratio=0.25
fraction=0
feasible profit=7.5
optimal solution is .....7.5_
```

Result:

~~17/3/23~~ Thus the ~~C++~~ program for Knapsack using greedy Technique has been Verified and executed successfully.

Exp. No: 05

SINGLE SOURCE SHORTEST PATH
USING GREEDY METHOD

Date:

17/3/23

Aim:

To write a C++ program to single source shortest path (Dijkstra's algorithm) using greedy technique and analyze the time and space complexity.

ALGORITHM:

§ for $i = 1$ to n do.

§ $S[i] = \text{false}$; $\text{dist}[i] := \text{cost}[V, i]$;

§ $S[V] = \text{true}$; $\text{dist}[V] = 0.0$;

§ put V in S

for $\text{num} = 2$ to n do.

choose u from among those vertices and not in S such that $\text{dist}[u]$ is minimum.

§ $S[u] = \text{true}$; // put u in S

§ update distance.

if $[\text{dist}[w] > \text{dist}[u] + \text{cost}[u, w]]$ then

$\text{dist}[w] := \text{dist}[u] + \text{cost}[u, w]$;

§.

NAME:KEERTHANAA B; REG.NO:21TD0035; YEAR/SEM/SEC:
Y2/S4/A

SINGLE SOURCE SHORTEST PATH

```
#include<iostream.h>
#include<conio.h>
class path
{
private:
int i,j,m,n,b[50],cost[20][20],k,d[50];
int u,v,w,num,s[20];
public:
void getdata();
void sp();
void display();
};
void path::getdata()
{
cout<<"\n***SINGLE SOURCE SHORTEST PATH***\n";
cout<<"\n enter the number of nodes:";
cin>>n;
cout<<"\n enter the cost\n";
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(i==j)
{
cost[i][j]=0;
}
else
{
cout<<"\n cost["<<i<<","<<j<<"]:";
cin>>cost[i][j];
}
}
}
cout<<"\n";
}
void path::sp()
{
k=1;
```




```
clrscr();
```

```
cout<<"\n the cost of vertices are:\n";
```

```
cout<<"\t";
```

```
for(i=1;i<=n;i++)
```

```
cout<<i<<"\t";
```

```
for(i=1;i<=n;i++)
```

```
cout<<"-----";
```

```
cout<<"\n";
```

```
for(i=1;i<=n;i++)
```

```
{
```

```
cout<<i<<"\t";
```

```
for(j=1;j<=n;j++)
```

```
{
```

```
cout<<cost[i][j]<<"\t";
```

```
}
```

```
cout<<"\n";
```

```
}
```

```
cout<<"\n enter the source vertex:";
```

```
cin>>v;
```

```
for(i=1;i<=n;i++)
```

```
{
```

```
s[i]=0;
```

```
d[i]=cost[v][i];
```

```
}
```

```
s[v]=1;
```

```
b[k++]=v;
```

```
d[v]=0;
```

```
for(num=2;num<=n;num++)
```

```
{
```

```
m=9999;
```

```
for(j=1;j<=n;j++)
```

```
if((m>d[j])&&(s[j]==0))
```

```
{
```

```
m=d[j];
```

```
u=j;
```

```
}
```

```
s[u]=1;
```

```
b[k++]=u;
```

```
for(w=1;w<=n;w++)
```

```
{
```

```
if(d[w]>(d[u]+cost[u][w])&&(s[w]==0))
```

```
{
```

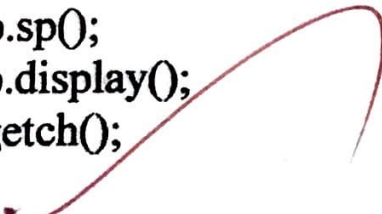
```
d[w]=(d[u]+cost[u][w]);
```

```
}
```




```
}

cout<<"\n";
}
}
void path::display()
{
cout<<"\n";
cout<<"\n the shortest path with source vertex"<<v<<"is \n";
cout<<"\n\t\t"<<b[1];
for(i=2;i<k;i++)
cout<<"----->"<<b[i];
}
void main()
{
clrscr();
path p;
p.getdata();
p.sp0;
p.display();
getch();
}
```



OUTPUT:

```
CATCUBMTC.DXE
***SINGLE SOURCE SHORTEST PATH***
enter the number of nodes:3
enter the cost
cost[1,2]:13
cost[1,3]:18
cost[2,1]:19
cost[2,3]:22
cost[3,1]:24
cost[3,2]:26
```

```
CATCUBMTC.DXE
the cost of vertices are:
1      2      3      -----
1      0      13     18
2      19     0      22
3      24     26     0

enter the source vertex:2

the shortest path with source vertex2is
2----->1----->3
```

Result:

17/3/23 Thus the C++ program for single source shortest path using greedy method is verified and executed successfully.

